SEN4100 – Software Project Management

Week 4 Lecture Notes: Project Planning, WBS & Software Cost Estimation

- **Course Learning Outcomes (CLOs) Addressed:**
  - Apply scheduling, budgeting, and resource-allocation techniques starting with scope decomposition.
  - Evaluate project performance using industry-standard metrics.
- **Primary Textbooks:** Pressman & Maxim (*Software Engineering: A Practitioner's Approach*), Hughes & Cotterell (*Software Project Management*).

  Note on Schedule Revision: Due to the Republic Day (October 29) national holiday, our active semester is adjusted to 12 lecture weeks. To ensure all fundamental planning concepts are covered before the Midterm Examination, this lecture integrates WBS decomposition with Cost & Effort estimation.

1. The Bridge from Scope to Planning

In Week 3, we defined the *boundaries* of the project through the Project Charter and the Scope Statement. Initiation answers the question: **"What are we building, and why?"**
Planning answers the questions: **"How will we build it? How long will it take? How much will it cost?"**
You cannot estimate costs or assign developers until you know exactly what tasks exist. Therefore, planning follows a strict two-step mathematical sequence:

1. **Decomposition:** Break the scope down into granular Work Packages via the WBS.
2. **Estimation:** Apply sizing and algorithmic cost models to those specific Work Packages.

PART I: THE WORK BREAKDOWN STRUCTURE (WBS)

2. Foundations of the WBS

The **Work Breakdown Structure (WBS)** is a hierarchical, deliverable-oriented decomposition of the total scope of work to be executed by the project team.

- **It is NOT a schedule:** A WBS does not show chronological sequence or dependencies.
- **It is NOT an organizational chart:** It breaks down the product and the engineering work, not the HR hierarchy.

3. The 100% Rule & Decomposition

The most critical design principle of WBS is the **100% Rule**:

The WBS must capture 100% of the work defined by the project scope, including internal project management activities, and it must capture nothing more.

If a task (e.g., "Write User Manual", "Provision AWS Servers") is not explicitly mapped to a WBS node, no budget or time will be allocated to it, guaranteeing an overrun schedule.

The Hierarchy Levels:

- **Level 1: The Project** (e.g., *E-Commerce App 1.0*)
- **Level 2: Control Accounts** (e.g., *1.0 Requirements, 2.0 Backend, 3.0 Frontend, 4.0 QA Testing*)
- **Level 3: Deliverables** (e.g., *2.1 Database Setup, 2.2 Payment API*)
- **Level 4/5: Work Packages** (e.g., *2.2.1 Configure OAuth, 2.2.2 Write JWT Logic*)

4. Anatomy of a Work Package & The WBS Dictionary

A **Work Package** is the lowest level of the WBS tree. It is sized using the **8/80 Rule**: A task should take no less than 8 hours and no more than 80 hours. If it takes longer than 80 hours (2 weeks), it is a "black box" and must be decomposed further.

To prevent ambiguity, every Work Package must be documented in the **WBS Dictionary**, which includes:

- Work Package Name & Description.
- Assigned Owner.
- **Estimated Cost & Effort** (Which leads us to Part II).
- Acceptance Criteria (Definition of Done).

PART II: COST & EFFORT ESTIMATION

Once the WBS provides us with distinct Work Packages, the Software Project Manager (SPM) must estimate the effort (person-months) and cost (budget) required to complete them.

5. Software Sizing Techniques

Before we can calculate time and money, we must calculate the *size* of the software to be built.

A. Lines of Code (LOC) Estimation:

- *Concept:* Estimating the size of a work package by predicting how many lines of source code will be written.
- *Pros:* Easy to measure once the project is finished.
- *Cons:* Highly dependent on the programming language (1 line of Python ≠ 1 line of Assembly). Punishes efficient developers who write concise, refactored code.

B. Function Point (FP) Analysis:

- *Concept:* Measures software size based on the functionality requested by and provided to the end-user, independent of the programming language.
- *Mechanics:* Evaluates the system across 5 information domain characteristics:

1. External Inputs (EI) - e.g., Data entry forms.
2. External Outputs (EO) - e.g., Generated reports, screens.
3. External Inquiries (EQ) - e.g., Search queries.

4. Internal Logical Files (ILF) - e.g., Local databases.
5. External Interface Files (EIF) - e.g., APIs pulling data from other systems.

- *Calculation:* These counts are weighted by complexity (Simple, Average, Complex) to calculate an Unadjusted Function Point (UFP) count, which is then adjusted by 14 Value Adjustment Factors (VAF) such as required transaction rates or distributed processing needs.

6. Algorithmic Cost Modeling: COCOMO

The **COnstructive COst MOdel (COCOMO)**, developed by Barry Boehm, is an algorithmic model that converts software size (KLOC - Thousands of Lines of Code) into estimated effort (Person-Months) and duration.

Basic COCOMO Formulas:

- **Effort**$(\boldsymbol{E}) = a_b \times (KLOC)^{b_b}$ [Measured in Person-Months]
- **Duration** $(\boldsymbol{D}) = c_b \times (E)^{d_b}$ [Measured in Months]
- **Staffing Required** $= E/D$ [Measured in Headcount]

Note: The coefficients (a_b, b_b, c_b, d_b) change based on the project mode:

1. **Organic Mode:** Small teams, familiar domain, flexible requirements (e.g., an internal HR tool).
2. **Semi-Detached Mode:** Medium teams, mixed experience, rigid and flexible requirements combined.
3. **Embedded Mode:** Complex, highly regulated, tight hardware constraints (e.g., the Smart Infusion Pump from Week 2).

7. Agile Estimation Techniques

Traditional models (COCOMO/FP) assume a predictive (Waterfall) lifecycle where requirements are locked. In Agile (Scrum/Kanban), we estimate value and complexity using relative sizing.

A. Story Points:

- A relative unit of measure representing the overall effort required to fully implement a User Story (Work Package).
- Story points encapsulate three factors: **Volume** (How much is there?), **Complexity** (How hard is it?), and **Uncertainty** (What are the risks?).
- Sized using the Fibonacci sequence (1, 2, 3, 5, 8, 13, 21...) because humans are better at estimating relative magnitudes than absolute hours.

B. Planning Poker:

- A consensus-based gamified technique. Developers anonymously select a Story Point card for a backlog item. If estimates diverge wildly (e.g., one dev plays a 3, another plays a 13), they debate the hidden complexities until consensus is reached. This eliminates "Anchoring Bias" from senior developers.

8. Alignment with Term Project: Milestones 1 & 2

This combined lecture sets up your project deliverables perfectly:

- **Milestone 1 Completion (Due Now):** Your squad must submit the visual WBS diagram (adhering to the 100% rule) and the WBS Dictionary for three core work packages.
- **Milestone 2 Preparation (Week 7 Checkpoint):** Now that you know *what* your work packages are, your next assignment will be to apply **Cost & Effort Estimation** to them. Your squad will need to estimate the work using either algorithmic models (COCOMO/Function Points) for predictive lifecycles, or Story Points for Agile lifecycles, and document these estimates in your Jira/GitHub board.

9. Classroom Discussion Scenarios

1. **The Estimation Trap:**
   - A corporate executive asks you: *"I need to know exactly how much this mobile app will cost and how many hours it will take before you do any WBS planning."*
   - *Question:* How do you respond to this using the concepts of the "Cone of Uncertainty"? Why is estimating without a WBS dangerous?
2. **LOC vs. Function Points:**
   - Two teams build the exact same software. Team A writes it in Assembly language and produces 100,000 LOC. Team B uses Python and produces 10,000 LOC.
   - *Question:* If you use LOC to estimate productivity, which team looks more productive? Which team delivered value faster? Why does this prove Function Points are a superior metric for client value?
3. **Planning Poker Dynamics:**
   - During a sprint planning session, a junior developer estimates a task at 13 Story Points, while the Senior Architect estimates it at 2 Story Points.
   - *Question:* Why should the Scrum Master encourage the junior developer to speak first during the Planning Poker debate? (Hint: Anchoring Bias and hidden knowledge).

10. Next Week Preview

- **Week 5 Topic:** Project Scheduling (Gantt, Ağ Diyagramları, PERT, CPM).
- **Core Agenda:** Now that we know *what* the tasks are (WBS) and *how long* they take (Estimation), we will map them against time. We will cover Activity Sequencing, Precedence Relationships (FS, SS, FF, SF), and calculate the Critical Path (CPM).
- **Preparation:** Please review the WBS you submitted for Milestone 1, as you will use those exact nodes to build your PERT network diagrams next week.